

Implementing A Simple RMAP Initiator Using The STAR-System APIs

STAR-Dundee's new software stack, STAR-System, provides a consistent programming interface for accessing STAR-Dundee interface and router devices, including the STAR-Dundee SpaceWire PCI Mk2, cPCI Mk2 and PCIe devices and the new STAR-Dundee SpaceWire Brick Mk2 and Router Mk2S devices.

The main API provided with STAR-System, STAR-API, provides functions for opening channels to devices and transmitting and receiving packets on those channels. One of the other APIs provided is the RMAP Packet Library, which provides functions for building Remote Memory Access Protocol (RMAP) packets, checking that a packet is a valid RMAP packet, and obtaining the values of the fields in an RMAP packet.

This application note explains how the RMAP Packet Library can be used to create RMAP packets, check the format of RMAP packets, and get the value of RMAP packet fields. These examples and the transmit and receive code demonstrated in the application note "**Transmitting And Receiving Packets Using STAR-API**" are then combined to demonstrate how a very basic RMAP Initiator can be implemented, which transmits an RMAP read command packet and receives an RMAP read reply packet. Further information on the APIs and the functions provided can be found in the STAR-System documentation.

Creating RMAP Packets

The RMAP Packet Library provides three types of functions for building RMAP packets:

- [`RMAP\_Calculate\*PacketLength\(\)`](#)
These functions can be used to calculate the length of a specific type of packet. An example of these functions is the [`RMAP\_CalculateWriteCommandPacketLength\(\)`](#) function, which calculates the length of an RMAP write command packet. Each of these functions takes as parameters the properties of the packet to be built which affect the packet's length, and returns the length of the packet. This length can then be used to allocate a buffer to store the packet, and the corresponding [`RMAP\_Fill\*Packet\(\)`](#) function called to store the packet in the buffer.
- [`RMAP\_Fill\*Packet\(\)`](#)
These functions can be used to create a specific type of RMAP packet, storing it in the buffer provided. An example of these functions is the [`RMAP\_FillWriteCommandPacket\(\)`](#) function which fills a buffer with an RMAP write command packet. The properties of the packet to be built are specified as parameters, along with the buffer to be filled and the length of the buffer. These functions are commonly used with the [`RMAP\_Calculate\*PacketLength\(\)`](#) functions, to first determine the length of the packet to be created, then allocate memory for the packet before filling the buffer.
- [`RMAP\_Build\*Packet\(\)`](#)
These functions combine the operation of the [`RMAP\_Calculate\*PacketLength\(\)`](#) and [`RMAP\_Fill\*Packet\(\)`](#) functions, while also allocating memory for the packet. An example is the [`RMAP\_BuildWriteCommandPacket\(\)`](#) which creates an RMAP write command packet. These functions calculate the amount of memory required for the packet, allocate memory for the packet, then fill that memory with the packet. The properties of the packet to be built are specified as parameters. When the buffer created by these functions is no longer required, it should be freed by calling [`RMAP\_FreeBuffer\(\)`](#).

Typically the `RMAP_Build*Packet()` functions are used to build RMAP packets. However, if you wish to allocate memory from a specific pool, or if you have some other form of memory allocation restrictions, the `RMAP_Calculate*PacketLength()` and `RMAP_Fill*Packet()` functions can be used. The example below shows how to use the `RMAP_CalculateReadCommandPacketLength()` and `RMAP_FillReadCommandPacket()` functions to create a read command packet:

```
void *pFillPacket;
unsigned long fillPacketLengthCalculated, fillPacketLength;
U8 targetAddress[] = {0, 254};
U8 replyAddress[] = {254};
const unsigned long targetAddressLength = sizeof(targetAddress);
const unsigned long replyAddressLength = sizeof(replyAddress);
char success;
const char alignment = 1, incrementAddress = 0;
const U8 key = 0x20, extendedReadAddress = 0;
const U16 transactionIdentifier = 0;
const U32 readAddress = 0x106, readLength = 4;

/* Calculate the length of a read command packet */
fillPacketLengthCalculated = RMAP_CalculateReadCommandPacketLength(
    targetAddressLength, replyAddressLength, alignment);

(void)printf("The read command packet will have a length of %ld bytes\n",
    fillPacketLengthCalculated);

/* Allocate memory for the packet */
pFillPacket = malloc(fillPacketLengthCalculated);
if (!pFillPacket)
{
    (void)puts("Couldn't allocate the memory for the read command packet");
    return;
}

/* Fill the memory with the packet */
success = RMAP_FillReadCommandPacket(targetAddress, targetAddressLength,
    replyAddress, replyAddressLength, incrementAddress, key,
    transactionIdentifier, readAddress, extendedReadAddress, readLength,
    &fillPacketLength, NULL, alignment, (U8 *)pFillPacket,
    fillPacketLengthCalculated);
if (!success)
{
    (void)puts("Couldn't fill the read command packet");
    free(pFillPacket);
    return;
}

/** \todo Do something with the packet */

/* Free the packet */
free(pFillPacket);
```

The example below shows how to use the `RMAP_BuildReadCommandPacket()` functions to achieve the same objective of creating a read command, with less code:

```

void *pBuildPacket;
unsigned long buildPacketLength;
U8 targetAddress[] = {0, 254};
U8 replyAddress[] = {254};
const unsigned long targetAddressLength = sizeof(targetAddress);
const unsigned long replyAddressLength = sizeof(replyAddress);
const char alignment = 1, incrementAddress = 0;
const U8 key = 0x20, extendedReadAddress = 0;
const U16 transactionIdentifier = 0;
const U32 readAddress = 0x106, readLength = 4;

/* Build the read command packet */
pBuildPacket = RMAP_BuildReadCommandPacket(targetAddress, targetAddressLength,
    replyAddress, replyAddressLength, incrementAddress, key,
    transactionIdentifier, readAddress, extendedReadAddress, readLength,
    &buildPacketLength, NULL, alignment);
if (!pBuildPacket)
{
    (void)puts("Couldn't build the read command packet");
    return;
}

/** \todo Do something with the packet */

/* Free the packet */
RMAP_FreeBuffer(pBuildPacket);

```

Checking the Format of RMAP Packets

The `RMAP_CheckPacketValid()` function can be used to check if a buffer contains a valid RMAP packet. The buffer and length of the buffer are passed as parameters, while a pointer to a previously allocated structure can also be passed in. If provided this structure will be updated to contain the properties of the RMAP packet (if valid). The structure can then be passed to the functions described in the following section to get the value of the fields in the RMAP packet. A final parameter passed to the function indicates whether to check if the packet is longer than expected. This can be used to check for unexpected bytes which follow the end of the RMAP packet. When the transmitter or receiver of the packet is only capable of sending packets that are multiples of 4 bytes in length, for example there may be additional padding bytes at the end of the packet and so it may be necessary to ignore these bytes.

The checks that are made by the `RMAP_CheckPacketValid()` function are listed below:

- The instruction field is valid:
 - The packet type is valid
 - The reserved bit of the packet type is 0
- The packet length is correct (unless this is not being checked)
- The protocol ID is the RMAP protocol ID
- The reply address (if expected) is present
- The data (if expected) is present
- In a read or read/modify/write reply, the reserved byte is 0
- The target and reply addresses are terminated with a logical address
- The header and (if present) data CRCs are correct
- In a read/modify write command, the data and mask length is 0, 2, 4, 6 or 8

Note that the `RMAP_CheckPacketValid()` function expects there to be one and only one address byte at the start of the packet. The function is not capable of looking through the start of the packet to determine where the address ends. When an RMAP packet reaches its destination, it should only have

one byte at the start, so this won't normally be a problem. However, passing a packet created with one of the `RMAP_Build*Packet()` functions, for example, will indicate the packet is invalid if the address specified was longer than one byte.

The example below builds a write register command packet (a 4-byte write command packet), and checks that it is valid. Note that the address at the start of the packet will contain two bytes, so the first byte is skipped when passing the packet to the `RMAP_CheckPacketValid()` function. This is the form the packet should be in when it reaches its destination:

```
void *pPacket;
U8 targetAddress[] = {0, 254};
U8 replyAddress[] = {254};
const unsigned long targetAddressLength = sizeof(targetAddress);
const unsigned long replyAddressLength = sizeof(replyAddress);
unsigned long packetLength;
RMAP_PACKET packetStruct;
RMAP_STATUS status;

/* Build an RMAP write command packet to write to a 4-byte register */
pPacket = RMAP_BuildWriteRegisterPacket(targetAddress, targetAddressLength,
    replyAddress, replyAddressLength, 1, 1, 0, 0x20, 0, 0x106, 0, 0x12345678,
    &packetLength, &packetStruct, 1);

/* Check the format of the packet, making sure it's not longer than expected. */
/* Note that the first byte is skipped as RMAP_CheckPacketValid() expects the */
/* address at the start to be 1 byte long. */
status = RMAP_CheckPacketValid((U8 *)pPacket + 1, packetLength - 1, &packetStruct,
    1);
(void)printf("The packet is %sa valid RMAP packet.\n",
    (status == RMAP_SUCCESS) ? "" : "not ");

/* Free the write command packet */
RMAP_FreeBuffer(pPacket);
```

Getting the Value of RMAP Packet Fields

To get the value of the fields in an RMAP packet, a number of `RMAP_Get*()` functions are provided. Each of these functions expects as a parameter a structure populated by the `RMAP_CheckPacketValid()` function. This structure will point to areas of the original packet buffer, so this buffer must also still exist when calling the `RMAP_Get*()` functions. The example below shows how to get the type, target address and key of an `RMAP_PACKET` structure previously obtained from a call to `RMAP_CheckPacketValid()`:

```

RMAP_PACKET packetStruct;
U8 *pAddress, key;
unsigned long addressLength, i;

/* Get the type of the packet */
switch (RMAP_GetPacketType(&packetStruct))
{
    case RMAP_WRITE_COMMAND:
        (void)puts("The packet is an RMAP write command.");
        break;

    case RMAP_WRITE_REPLY:
        (void)puts("The packet is an RMAP write reply.");
        break;

    case RMAP_READ_COMMAND:
        (void)puts("The packet is an RMAP read command.");
        break;

    case RMAP_READ_REPLY:
        (void)puts("The packet is an RMAP read reply.");
        break;

    case RMAP_READ_MODIFY_WRITE_COMMAND:
        (void)puts("The packet is an RMAP read/modify/write command.");
        break;

    case RMAP_READ_MODIFY_WRITE_REPLY:
        (void)puts("The packet is an RMAP read/modify/write reply.");
        break;

    case RMAP_INVALID_PACKET_TYPE:
        (void)puts("The packet is of an invalid RMAP packet type.");
        break;

    default:
        (void)puts("The packet type could not be obtained.");
}

/* Get the target address of the packet */
if ((pAddress != NULL) && (addressLength))
{
    /* Display the target address of the packet */
    (void)printf("The packet has a target address of:");
    for (i = 0; i < addressLength; i++)
    {
        (void)printf(" %x", pAddress[i]);
    }
    (void)puts(".");
}
else
{
    (void)puts("No valid target address is present in the packet.");
}

/* Get the key of the packet */
key = RMAP_GetKey(&packetStruct);
(void)printf("The value of the key is 0x%x.\n", key);

```

RMAP Initiator Example Code

The examples above can be used to create an RMAP command packet to be transmitted, to check that a received packet is a valid RMAP packet, and to get the values of the fields in that packet. When combined with the information in the “**Transmitting and Receiving Packets Using STAR-API**” application note, this is all the information required to write a simple RMAP Initiator.

The code below builds an RMAP read command packet and transmits it. It then waits for a reply packet to be received. If the reply packet is a valid RMAP packet, is a read reply, and is the reply for the command that was sent (checked using the transaction identifier), then the data read is displayed.

```

STAR_DEVICE_ID *pDeviceList, deviceId;
U32 deviceCount;
STAR_CHANNEL_MASK channelMask;
STAR_CHANNEL_ID channelId;
STAR_STREAM_ITEM *pTxStreamItem, *pRxStreamItem;
STAR_TRANSFER_OPERATION *pRxTransOp, *pTxTransOp;
STAR_TRANSFER_STATUS receiveStatus;
void *pCommandPacket, *pReplyPacket;
unsigned long commandPacketLength;
unsigned int replyPacketLength, packetCount;
U8 targetAddress[] = {0, 254};
U8 replyAddress[] = {254};
const unsigned long targetAddressLength = sizeof(targetAddress);
const unsigned long replyAddressLength = sizeof(replyAddress);
const char alignment = 1, incrementAddress = 1;
const U8 key = 0x20, extendedReadAddress = 0;
const U16 transactionIdentifier = 0x1234;
U16 transactionIdentifierReceived;
const U32 readAddress = 0x106, readLength = 4;
RMAP_STATUS status;
RMAP_PACKET packetStruct;
U8 *pData;
const char channelNumber = 1;
U32 dataLength, i;

/* Get the list of devices */
pDeviceList = STAR_getDeviceList(&deviceCount);

/* Check there are devices present */
if (!pDeviceList)
{
    (void)puts("No devices present.");
    return;
}
if (!deviceCount)
{
    (void)puts("No devices present.");
    STAR_destroyDeviceList(pDeviceList);
    return;
}

(void)printf("%d devices present.\n", deviceCount);

/* Store the device identifier of the first device */
deviceId = pDeviceList[0];

/* Destroy the device list */
STAR_destroyDeviceList(pDeviceList);

```



```

/* Get the channels on the device and check that the channel to be used exists */
channelMask = STAR_getDeviceChannels(deviceId);
if (!(channelMask & (channelNumber << 1)))
{
    (void)printf("The device in use doesn't have a channel %d.", channelNumber);
    return;
}

/* Open a bidirectional channel to channel 1 of the device */
channelId = STAR_openChannelToLocalDevice(deviceId, STAR_CHANNEL_DIRECTION_INOUT,
    channelNumber, 1);
if (!channelId)
{
    (void)puts("Unable to open a channel to the device.");
    return;
}

/* Build an RMAP read command packet to read 4 bytes */
pCommandPacket = RMAP_BuildReadCommandPacket(targetAddress, targetAddressLength,
    replyAddress, replyAddressLength, incrementAddress, key,
    transactionIdentifier, readAddress, extendedReadAddress, readLength,
    &commandPacketLength, NULL, alignment);
if (!pCommandPacket)
{
    (void)puts("Couldn't build the read command packet");
    goto buildPacketFailed;
}

/* Create a packet stream item for the RMAP read command to be transmitted, */
/* terminated with an EOP */
pTxStreamItem = STAR_createPacket(NULL, (unsigned char *)pCommandPacket,
    commandPacketLength, STAR_EOP_TYPE_EOP);
if (!pTxStreamItem)
{
    (void)puts("Couldn't create the stream item to be transmitted.");
    goto createPacketFailed;
}

/* Create a transfer operation to transmit the packet */
pTxTransOp = STAR_createTxOperation(&pTxStreamItem, 1);
if (!pTxTransOp)
{
    (void)puts("Couldn't create the transmit operation.");
    goto createTxOpFailed;
}

/* Create a transfer operation for receiving one packet, the RMAP reply */
pRxTransOp = STAR_createRxOperation(1, STAR_RECEIVE_PACKETS);
if (!pRxTransOp)
{
    (void)puts("Couldn't create receive operation, exiting.");
    goto createRxOpFailed;
}

/* Start transmitting the packet */
(void)STAR_submitTransferOperation(channelId, pTxTransOp);

/* Start receiving a packet */
(void)STAR_submitTransferOperation(channelId, pRxTransOp);

```

```

/* Wait indefinitely on the reply packet being received */
receiveStatus = STAR_waitOnTransferOperationCompletion(pRxTransOp, -1);
if (receiveStatus != STAR_TRANSFER_STATUS_COMPLETE)
{
    (void)printf("Could not receive packet, error of %d.\n", receiveStatus);
    goto rxFailed;
}

/* Get the stream item received */
pRxStreamItem = STAR_getTransferItem(pRxTransOp, 0);
if (!pRxStreamItem)
{
    (void)puts("Unable to get the stream item received.");
    goto rxFailed;
}

/* Get the bytes of the packet received */
pReplyPacket = STAR_getPacketData(
    (STAR_SPACEWIRE_PACKET *)pRxStreamItem->item, &replyPacketLength);
if (!pReplyPacket)
{
    (void)puts("Unable to get the contents of the packet.");
    goto rxFailed;
}

/* Check the format of the received packet is a valid RMAP packet */
status = RMAP_CheckPacketValid((U8 *)pReplyPacket, replyPacketLength,
    &packetStruct, 1);
if (status != RMAP_SUCCESS)
{
    (void)puts("The packet received is not a valid RMAP packet.");
    goto rmapFailed;
}

/* Check the packet is a read reply */
if (RMAP_GetPacketType(&packetStruct) != RMAP_READ_REPLY)
{
    (void)puts("The packet received is not an RMAP read reply packet.");
    goto rmapFailed;
}

/* Check the status in the reply packet is success */
status = RMAP_GetStatus(&packetStruct);
if (status != RMAP_SUCCESS)
{
    (void)puts("The reply packet contains an error status.");
    goto rmapFailed;
}

/* Check the transaction identifier is correct */
transactionIdentifierReceived = RMAP_GetTransactionID(&packetStruct);
if (transactionIdentifierReceived != transactionIdentifier)
{
    (void)puts("The packet received has an unexpected transaction identifier.");
    (void)printf("Expected a value of 0x%x, received a value of 0x%x\n",
        transactionIdentifier, transactionIdentifierReceived);
    goto rmapFailed;
}

```



```

/* Get the data in the reply */
pData = RMAP_GetData(&packetStruct, &dataLength);
if (!pData)
{
    (void)puts("Unable to get the data in the RMAP read reply packet.");
    goto rmapFailed;
}
if (dataLength != readLength)
{
    (void)puts("Unexpected data length in the RMAP read reply packet.");
    (void)printf("Expected a length of %d, received a length of %d\n", dataLength,
        readLength);
}

/* Display the data in the reply */
(void)printf("The data read:");
for (i = 0; i < dataLength; i++)
{
    (void)printf(" %x", pData[i]);
}
(void)puts(".");

rmapFailed:
/* Destroy the buffer containing the received packet */
STAR_destroyPacketData((unsigned char *)pReplyPacket);

rxFailed:
/* Dispose the receive operation */
(void)STAR_disposeTransferOperation(pRxTransOp);

createRxOpFailed:
/* Dispose the transmit operation */
(void)STAR_disposeTransferOperation(pTxTransOp);

createTxOpFailed:
/* Destroy the transmit packet */
STAR_destroyStreamItem(pTxStreamItem);

createPacketFailed:
/* Free the read command packet */
RMAP_FreeBuffer(pCommandPacket);

buildPacketFailed:
/* Close the channel to the device */
STAR_closeChannel(channelId);

```

Note that it is possible to simplify this code using the `STAR_transmitPacket()` and `STAR_receivePacket()` functions. This example does not make use of these functions as they are only useful in certain situations. Using `STAR_createTxOperation()`, `STAR_createRxOperation()` and `STAR_submitTransferOperation()`, for example, is more flexible and efficient.

